MiroFish 项目亮点、难点与后续项目参考

1. 项目定位

MiroFish 的核心价值，不是单点能力，而是把一条完整链路跑通：

`非结构化材料 -> 本体 -> 图谱 -> Agent 人设 -> 仿真环境 -> 报告生成 -> 深度交互`

和很多只做到“知识抽取”或“多 Agent 对话”的项目不同，MiroFish 更接近一个可运行的数字世界构建系统。它把现实材料转换成结构化世界模型，再让这个世界可以运行、演化、被检索、被总结、被追问。

2. 项目亮点

2.1 完整闭环

项目实现了从输入材料到最终报告与交互的完整产品闭环：

- 上传种子材料
- 自动生成本体
- 自动构建图谱
- 自动筛选实体并生成人设
- 自动生成仿真参数
- 启动双平台模拟
- 自动生成预测报告
- 支持继续追问和采访 Agent

这意味着它不是单个算法 Demo，而是具备系统交付形态。

2.2 模块拆分清晰

后端核心服务划分比较合理：

- `ontology_generator.py`：本体生成
- `graph_builder.py`：图谱注册和文本入图
- `zep_entity_reader.py`：读取和筛选图谱实体
- `oasis_profile_generator.py`：实体到 Agent 人设转换
- `simulation_config_generator.py`：仿真参数生成
- `simulation_manager.py` / `simulation_runner.py`：仿真准备与执行
- `zep_graph_memory_updater.py`：模拟过程反写图谱
- `report_agent.py`：报告生成与工具调用
- `zep_tools.py`：GraphRAG 检索和分析工具集

这种分层便于后续替换某个能力，而不必推翻整套系统。

2.3 Schema First 设计

项目先用 LLM 生成“本体”，再把本体交给图服务去做抽取。

PROF

这个思路的优点是：

- 避免直接手写大量实体/关系规则
- 可以针对不同领域快速迁移
- 后续图谱抽取有明确约束，不是完全开放式抽取

这是一个很值得复用的设计。

2.4 动态时序 GraphRAG

这个项目不是只在初始阶段构图，而是把模拟过程中的行为继续写回图谱，形成动态的时序记忆。

它带来的价值是：

- 图谱不仅描述初始现实材料
- 图谱还能描述模拟世界的演化过程
- 后续报告和分析可以同时使用“初始事实 + 演化后的事实”

这比静态 GraphRAG 更适合推演类系统。

2.5 报告生成不是单次生成，而是工具增强链式生成

ReportAgent 的设计是项目中很强的一部分。它不是“一次 prompt 生成全文”，而是：

- 先规划大纲
- 再逐章节生成
- 每章中间可调用多种检索工具
- 信息不足时继续检索
- 最后输出章节内容并拼装整篇报告

这是典型的“链式推理 + 工具增强 + 分阶段落地”，稳定性比单次长文本生成更好。

2.6 有实用的降级与容错设计

项目中已经出现不少很实用的工程防线，例如：

- LLM JSON 解析失败修复
- 输出截断修复
- 多次重试
- LLM 失败时使用默认配置
- 人设生成支持规则 fallback
- 异步任务进度保存

这些设计说明项目已经在真实运行中遇到过不稳定问题，并做了实际处理。

2.7 项目是如何进行 LLM 交互的

这个项目的 LLM 不是”问一句答一句”，而是嵌在一条流水线里：

基本流程：业务模块构造 prompt → 调用 LLM → 拿到 JSON 结果 → 交给下一步继续用

PROF

LLM 在这里主要是"结构化决策器"和"分析规划器"，不是简单的文本生成器。

统一客户端：`backend/app/utils/llm_client.py`

- `chat_json()` 方法：要求输出 JSON（本体、配置、子问题拆解）
- `chat()` 方法：要求输出自然语言（报告章节、总结、对话）

2.7.1 直接调用 LLM 的 5 个关键步骤

1. 本体生成

- 输入：模拟需求 + 文档文本
- 输出：`entity_types`、`edge_types`、`analysis_summary`
- 代码位置：`backend/app/services/ontology_generator.py` 的 `generate()` 方法

2. Agent 人设生成

- 输入：实体名称、类型、摘要、关联事实、Zep 检索上下文
- 输出：`bio`、`persona`、兴趣、职业、年龄等
- 支持规则 fallback（LLM 失败时用默认规则）
- 代码位置：`backend/app/services/oasis_profile_generator.py` 的 `_generate_profile_with_llm()` 方法

3. 仿真配置生成

- 分多步调用：时间配置 → 事件配置 → Agent 行为参数
- 输出：模拟总时长、高峰时段、初始帖子、Agent 活跃度等
- 失败时有默认配置兜底
- 代码位置：`backend/app/services/simulation_config_generator.py` 的 `generate_config()` 方法

4. 报告生成（最复杂）

- 流程：规划大纲 → 逐章节生成 → 调用 GraphRAG 工具 → 产出最终报告
- 使用 ReACT 循环：LLM 决定什么时候调工具、调哪个工具
- 代码位置：`backend/app/services/report_agent.py`
  - `plan_outline()` 方法：规划大纲
  - `_generate_section_react()` 方法：ReACT 循环生成章节

5. GraphRAG 辅助推理

- 拆解复杂问题成多个子问题
- 选择采访对象、生成采访问题、汇总摘要
- 代码位置：`backend/app/services/zep_tools.py` 的 `insight_forge()` 方法

2.7.2 不直接调 LLM、但用了 LLM 产物的步骤

图谱构建：

- 本地代码把 LLM 生成的本体注册到 Zep

PROF

- 把文本块批量发给 Zep
- Zep 根据本体做实体和关系抽取（这一步不是本地 LLM 做的）
- 代码位置：`backend/app/services/graph_builder.py` 的 `set_ontology()` 和 `add_text_batches()` 方法

重要边界：

- 本地代码：定义 prompt、发起调用、解析结果、做 fallback
- Zep 服务：图谱抽取（黑盒依赖）

2.7.3 LLM 交互的特点

- **不是单点调用**，而是多阶段流水线
- **偏结构化输出**，不是自由生成
- **每个阶段都转成 JSON**，便于下一步消费
- **有 prompt chaining**，一步的输出是下一步的输入
- **有 fallback 机制**，不依赖单次调用一定成功
- **有明确的责任边界**：LLM 决策 + 本地执行 + 外部系统抽取

这个边界非常重要，因为它决定了系统的可解释性和调试方式。

2.7.5 项目的 LLM 交互特点

综合来看，这个项目的 LLM 交互有几个明显特点：

- 不是单点调用，而是多阶段流水线调用
- 更偏结构化输出，而不是自由生成
- 每个阶段都尽量把输出转成 JSON
- 存在明显的 prompt chaining 设计
- 存在 fallback 和默认配置机制
- 存在“LLM 决策，本地执行，外部系统抽取”的协作边界

2.7.6 对后续项目的启发

PROF

如果后续项目要借鉴这套 LLM 交互方式，建议保留以下原则：

- 让 LLM 负责高层语义决策，不要让它直接承担所有执行逻辑
- 关键阶段尽量输出结构化 JSON，而不是纯自然语言
- 把多步复杂任务拆成多个 prompt 阶段
- 为每一步设计 fallback，不依赖单次调用一定成功
- 区分“LLM 生成 schema”和“系统根据 schema 执行”

3. 项目难点

3.1 本体质量决定全局质量

本体生成是整条链的起点，也是最关键的放大器。

如果本体中的实体类型、边类型、属性定义、source-target 关系不准确，后面所有步骤都会受影响：

- 图谱抽取会歪
- 实体筛选会歪
- 人设生成会歪
- 模拟配置会歪
- 报告结论也会歪

因此，本体并不是“一个普通前处理步骤”，而是全链路的核心控制点。

3.2 外部依赖多，链路复杂

项目依赖多个系统共同工作：

- LLM 接口
- Zep 图服务
- OASIS 仿真系统
- 前端流程编排
- 后端异步任务

这种架构的好处是能力强，但工程难点也很明显：

- 任一外部服务超时都会影响全链路
- 某一环失败时，错误定位不直观
- 本地代码和外部平台的责任边界需要特别清楚

3.3 图谱抽取部分是黑盒依赖

本地代码负责：

- 定义本体
- 把本体提交给 Zep
- 把文本分批送入 Zep

但实体和边的实际抽取发生在 Zep 侧。

这意味着：

- 开发效率高
- 但可解释性降低
- debug 深度受限
- 精细控制边抽取策略较难

如果后续项目需要更强可控性，这会是一个重点问题。

3.4 仿真真实性与可控性之间存在天然冲突

人设越自由、行为越开放，越可能出现“看起来真实”的涌现行为；但同时：

- 结果更不稳定
- 更难复现
- 更难评估
- 更难调试

PROF

如果为了稳定性加入太多规则，模拟又容易变成模板化。

这是所有多 Agent 仿真系统都必须面对的核心矛盾。

3.5 报告链条长，成本和时延高

报告系统中可能涉及：

- 大纲生成
- 多章节多轮迭代
- 多次工具调用
- 多次图检索
- 采访 Agent
- 最终汇总

优点是质量更稳，缺点是：

- 耗时高
- API 成本高
- 对缓存和可观测性要求高

如果用于线上正式产品，必须从设计上考虑预算和延迟控制。

3.6 缺少统一评测标准是大问题

这类系统很难像普通问答那样只用一个 benchmark 来衡量。

至少要分层评测：

- 本体质量
- 实体抽取质量
- 边关系质量
- 人设合理性
- 仿真行为合理性
- 报告可信度与可解释性

PROF

如果没有分层评测，系统很容易出现“整体看起来挺厉害，但不知道哪里不对”的情况。

4. 对后续项目最有参考价值的设计

4.1 一定要保留分阶段中间产物

建议所有关键阶段都落盘保存：

- `ontology.json`
- `graph_snapshot.json`
- `profiles.json`
- `simulation_config.json`
- `report_outline.json`
- `section_logs.jsonl`

好处是：

- 可 debug
- 可回放
- 可比较不同版本输出
- 可做回归测试

4.2 一定要把“直接调用 LLM”和“使用 LLM 产物”分开

这个项目整体上已经有这个雏形。

后续项目建议更明确地区分：

- 哪些步骤是真正的 LLM 决策步骤
- 哪些步骤只是消费上一步 LLM 产物
- 哪些步骤是规则处理
- 哪些步骤是外部服务处理

这样能极大提升可解释性和排障效率。

4.3 报告生成优先采用分阶段链式生成

不要默认用“一次 prompt 写整篇报告”的方式。

更推荐：

- 先规划
- 再分章节
- 再结合检索工具
- 最后统一后处理

这种结构更适合复杂分析类产品，也更容易控制质量。

4.4 动态图记忆值得复用

PROF

如果后续项目也涉及：

- 多轮演化
- 长期状态更新
- 历史过程追踪
- 预测推演

那么“模拟行为反写图谱”是很值得继承的设计，而不应该只做一次性静态构图。

4.5 必须从一开始就设计降级机制

建议任何后续项目都默认支持：

- `use_llm` 开关
- 规则 fallback
- 默认配置 fallback

- 重试机制
- 部分结果可继续推进

原因很简单：LLM 和外部服务永远不可能稳定到完全不用兜底。

5. 如果重做一版，建议补强的能力

5.1 Prompt 版本管理

建议把关键 prompt 单独版本化，至少做到：

- prompt 有独立文件
- prompt 有版本号
- prompt 变更可对比
- prompt 可绑定评测样本

否则后期很容易出现“效果变了，但不知道是哪段 prompt 导致”的问题。

5.2 全链路 Trace

建议给每个任务分配统一 trace id，并串联：

- API 请求
- LLM 调用
- Zep 调用
- 仿真轮次
- 报告生成步骤

这样定位问题会快很多。

5.3 更清晰的数据协议

建议统一定义关键 JSON schema，确保不同模块可以解耦替换。

例如：

- ontology schema
- entity schema
- edge schema
- profile schema
- simulation config schema
- report outline schema

这会显著提升系统长期可维护性。

5.4 离线回放能力

建议支持只重跑某个阶段，例如：

- 只重跑本体
- 只重跑图谱构建

PROF

- 只重跑人设生成
- 只重跑报告

不要要求每次都从头执行整条链。

5.5 分层评测体系

建议建立最小评测集，至少覆盖：

- 本体定义是否合理
- 关系是否完整
- Agent 是否符合角色
- 模拟输出是否和预期方向一致
- 报告是否引用到了正确事实

没有评测，系统只能靠主观感觉优化。

6. 后续项目推荐的落地路径

阶段一：先做结构化世界

目标：

- 跑通文档解析
- 跑通本体生成
- 跑通图谱构建

先不要急着上复杂仿真。

阶段二：再做 Agent 化

目标：

- 从图谱筛出实体
- 生成稳定的人设
- 生成基础模拟参数

这一步主要验证“世界里的人是否成立”。

阶段三：再做演化与记忆更新

目标：

- 让 Agent 真正运行
- 让行为进入时序图谱
- 让检索能看到演化信息

这一步决定系统是否只是静态分析器，还是动态推演器。

阶段四：最后做报告与交互

PROF

目标：

- 用工具增强 Agent 生成报告
- 支持继续追问
- 支持采访模拟体

这样最终交付的就不是一个后台引擎，而是一个完整的可消费产品。

7. 核心技术难点深度解析

7.1 Zep 本体动态生成的技术挑战

问题：Zep SDK 要求用 Pydantic 模型定义本体，但本体是 LLM 动态生成的，没法提前写死类定义。

怎么解决的：

- 用 Python 的 `type()` 函数在运行时动态创建类
- 处理 Zep 的保留字段冲突（`uuid`, `name` 这些不能直接用，要改名）
- 给每个实体类型和关系类型都动态生成对应的 Pydantic 模型

代码位置：

- `backend/app/services/graph_builder.py` 的 `set_ontology()` 方法（199-286 行）

7.2 双平台并行模拟的资源管理

问题：

- Twitter 和 Reddit 两个平台同时跑，都要调 LLM API，容易被限流
- 模拟跑了几小时，突然被 Ctrl+C 打断，数据库可能锁死
- Windows 上中文乱码（OASIS 库读文件没指定编码）

怎么解决的：

- **双 LLM 配置**：Twitter 用一个 API，Reddit 用另一个 API（如果配了的话），分散压力
- **限流控制**：OASIS 环境设置 `semaphore=30`，最多同时 30 个 LLM 请求
- **优雅退出**：捕获 SIGTERM/SIGINT 信号，通知 asyncio 循环正常退出，不强杀
- **编码修复**：在所有 import 之前 Monkey-patch `builtins.open()`，强制用 UTF-8

代码位置：

- `backend/scripts/run_parallel_simulation.py`
  - `create_model()` 函数：双 LLM 配置（984-1037 行）
  - `setup_signal_handlers()` 函数：信号处理（1653-1682 行）
  - 文件开头：Windows 编码修复（29-65 行）

7.3 ReACT 循环的 LLM 行为控制

问题：LLM 不听话，经常出幺蛾子：

- 同时输出工具调用和最终答案（应该二选一）

PROF

- 不调工具直接写报告（应该先调工具收集信息）
- 调工具次数不够（要求至少 3 次，它只调 1 次）
- 输出裸 JSON 不包 `<tool_call>` 标签（解析失败）

怎么解决的：

- **冲突检测**：发现同时输出工具和答案，前两次拒绝要求重来，第三次强制截断只执行工具
- **最少调用限制**：工具调用不到 3 次不让输出答案，继续要求调工具
- **最大调用限制**：调了 5 次工具后强制输出答案，别再调了
- **裸 JSON 兜底**：检测到 `{` 开头 `}` 结尾的响应，尝试直接解析 JSON

代码位置：

- `backend/app/services/report_agent.py`
  - `_generate_section_react()` 方法：ReACT 循环主逻辑（1220-1530 行）
  - `_parse_tool_calls()` 方法：工具调用解析（1066-1124 行）

7.4 报告格式控制与后处理

问题：LLM 生成的报告格式乱七八糟：

- 重复标题（章节标题写了两遍）
- 引用格式不对（引用混在段落里，不独立成段）
- 标题层级混乱（应该用粗体的地方用了 ### 标题）

怎么解决的：

- **标题清理**：正则匹配所有 Markdown 标题，### 及以下级别全转成粗体
- **重复检测**：检查前 5 行有没有相同标题，有就删掉
- **引用格式强制**：在 Prompt 里明确要求引用必须独立成段，前后各空一行

代码位置：

- `backend/app/services/report_agent.py`
  - `_clean_section_content()` 方法：章节内容清理（2131-2196 行）
  - `_post_process_report()` 方法：整个报告后处理（2300-2423 行）
  - `SECTION_SYSTEM_PROMPT_TEMPLATE` 常量：Prompt 中的格式要求（614-766 行）

7.5 数据库增量读取与上下文补充

问题：

- Twitter 和 Reddit 的 `created_at` 格式不一样（一个是整数时间戳，一个是日期字符串）
- 动作日志只有 ID，没有完整信息（比如点赞帖子，只有 post_id，没有帖子内容）
- 每次都全量读数据库太慢，要增量读

怎么解决的：

- **用 rowid 追踪**：SQLite 的 `rowid` 是自增的，用它来追踪读到哪了，不用 `created_at`
- **上下文补充**：读到动作后，根据 ID 去关联查询帖子内容、用户名等完整信息
- **关联查询**：JOIN post 表、user 表，一次性拿到所有需要的信息

PROF

代码位置：

- `backend/scripts/run_parallel_simulation.py`
  - `fetch_new_actions_from_db()` 函数：增量读取（657-746 行）
  - `_enrich_action_context()` 函数：上下文补充（749-854 行）
  - `_get_post_info()` 等辅助函数：关联查询（857-981 行）

7.6 IPC 进程间通信设计

问题：

- 模拟进程跑在后台，API 进程要能控制它（比如采访 Agent）
- 不能用网络通信（太重），也不能用共享内存（太复杂）
- 要支持批量采访、远程关闭环境等命令

怎么解决的：

- **文件系统 IPC**：用文件夹当消息队列
  - API 进程写命令到 `ipc_commands/cmd_xxx.json`
  - 模拟进程轮询读命令，执行后写响应到 `ipc_responses/cmd_xxx.json`
  - 环境状态写到 `env_status.json`
- **命令轮询**：模拟进程每 0.5 秒检查一次有没有新命令
- **双平台采访**：用 `asyncio.gather()` 并行采访 Twitter 和 Reddit

代码位置：

- `backend/scripts/run_parallel_simulation.py`
  - `ParallelIPCHandler` 类：IPC 处理器（217-601 行）
  - `process_commands()` 方法：命令轮询（560-601 行）
  - `handle_interview()` / `handle_batch_interview()` 方法：采访处理（345-515 行）

8. 性能优化经验总结

8.1 并行化策略

PROF

/compact

图谱构建：

- 文本分块后批量发送（每批 3 个块），避免单个请求太大超时
- 代码位置：`backend/app/services/graph_builder.py` 的 `add_text_batches()` 方法

模拟运行：

- Twitter 和 Reddit 用 `asyncio.gather()` 并行跑，不用等一个跑完再跑另一个
- 双 LLM 配置：两个平台用不同的 API 服务商，提高并发能力
- 代码位置：`backend/scripts/run_parallel_simulation.py` 的 `main()` 函数（1585-1589 行）

Profile 生成：

- 支持并行生成（默认 3 个一起生成），不用一个一个串行

- 实时保存到文件，避免内存占用太大
- 代码位置：`backend/app/services/oasis_profile_generator.py` 的 `generate_profiles_from_entities()` 方法

8.2 缓存与增量更新

数据库增量读取：

- 用 `rowid` 记住读到哪了，下次只读新的，不重复读
- 代码位置：`backend/scripts/run_parallel_simulation.py` 的 `fetch_new_actions_from_db()` 函数

分章节保存：

- 每个章节写完立刻存文件，前端可以实时看到进度
- 不用等整个报告写完才能看
- 代码位置：`backend/app/services/report_agent.py` 的 `generate_report()` 方法（1636-1686 行）

Zep API 重试：

- API 调用失败自动重试 3 次，每次等待时间翻倍（指数退避）
- 避免偶尔的网络抖动导致整个流程失败
- 代码位置：`backend/app/services/zep_tools.py` 的 `_call_with_retry()` 方法

8.3 资源限制

LLM 并发控制：

- OASIS 环境设置 `semaphore=30`，最多同时 30 个 LLM 请求
- 避免一下子发太多请求被 API 限流
- 代码位置：`backend/scripts/run_parallel_simulation.py` 的 `run_twitter_simulation()` 和 `run_reddit_simulation()` 函数

PROF

日志优化：

- 禁用 OASIS 的冗余日志（每个 agent 的观察和动作都记录，太多了）
- 只用自定义的 action_logger 记录关键动作
- 代码位置：`backend/scripts/run_parallel_simulation.py` 的 `disable_oasis_logging()` 函数

内存管理：

- Profile 生成支持实时保存，不在内存里堆积
- 报告生成分章节落盘，不一次性加载全部内容
- 避免大文件一次性读到内存

---

9. 给后续项目的建议

9.1 一定要保留中间产物

为什么：

- 方便 debug（哪一步出问题了一眼就看出来）
- 可以回放（不用每次都从头跑）
- 可以对比不同版本的输出（改了 prompt 效果变好还是变差）

建议保存的文件：

- `ontology.json` - 本体定义
- `graph_snapshot.json` - 图谱快照
- `profiles.json` - Agent 人设
- `simulation_config.json` - 模拟配置
- `report_outline.json` - 报告大纲
- `section_logs.jsonl` - 章节生成日志

代码参考：

- `backend/app/models/project.py` - 项目状态管理
- `backend/app/services/report_agent.py` - 报告中间产物保存

9.2 区分"LLM 决策"和"使用 LLM 产物"

为什么：

- 提升可解释性（知道哪一步是 LLM 做的决定）
- 方便排查问题（LLM 出错还是代码逻辑出错）
- 便于替换组件（比如换个 LLM 或者换个图谱服务）

怎么做：

- 明确标记哪些步骤是真正调用 LLM 的
- 哪些步骤只是消费上一步 LLM 的输出
- 哪些步骤是纯规则处理
- 哪些步骤是外部服务处理（比如 Zep 图谱抽取）

PROF

代码参考：

- 本文档第 2.7 节"项目是如何进行 LLM 交互的"

9.3 报告生成优先用分阶段链式生成

为什么：

- 一次性生成整篇报告容易失控（太长、格式乱、质量不稳定）
- 分阶段生成可以每一步都检查质量
- 结合检索工具可以保证内容有依据

推荐流程：

1. 先规划大纲
2. 再分章节生成

3. 每章节中间调用检索工具
4. 最后统一后处理（清理格式）

代码参考：

- `backend/app/services/report_agent.py` 的 `generate_report()` 方法

9.4 动态图记忆值得复用

适用场景：

- 多轮演化的系统
- 需要追踪历史过程的系统
- 预测推演类系统

核心思路：

- 不只是初始构图，模拟过程中的行为也写回图谱
- 图谱既描述初始现实，也描述演化过程
- 后续分析可以同时使用"初始事实 + 演化后的事实"

代码参考：

- `backend/app/services/zep_graph_memory_updater.py` - 图谱更新服务

9.5 从一开始就设计降级机制

为什么：

- LLM 和外部服务永远不可能 100% 稳定
- 没有降级机制，一个环节失败整个流程就挂了

建议支持：

- `use_llm` 开关（可以关掉 LLM 用规则）
- 规则 fallback（LLM 失败时用默认规则）
- 默认配置 fallback（配置生成失败时用默认配置）
- 重试机制（API 调用失败自动重试）
- 部分结果可继续（不是全有全无）

代码参考：

- `backend/app/services/oasis_profile_generator.py` - Profile 生成支持规则 fallback
- `backend/app/services/zep_tools.py` - Zep API 调用支持重试

---

10. 如果重做，建议补强的能力

10.1 Prompt 版本管理

现状问题：

PROF

- Prompt 都写在代码里，改了不知道改了啥
- 效果变了，不知道是哪段 prompt 导致的

建议做法：

- Prompt 单独放文件，不混在代码里
- 每个 Prompt 有版本号
- Prompt 变更可以对比（像 git diff 一样）
- Prompt 可以绑定评测样本（改了 prompt 跑一遍测试）

10.2 全链路 Trace

现状问题：

- 出问题了不知道是哪个环节出的
- 日志散落在各个文件，串不起来

建议做法：

- 给每个任务分配统一的 trace_id
- 串联所有环节：API 请求、LLM 调用、Zep 调用、仿真轮次、报告生成
- 可以根据 trace_id 快速定位问题

10.3 更清晰的数据协议

现状问题：

- 不同模块之间的数据格式不够统一
- 想替换某个模块比较困难

建议做法：

- 统一定义关键 JSON schema
- 比如：ontology schema、entity schema、edge schema、profile schema、simulation config schema、report outline schema
- 模块之间只通过 schema 约定的格式交互

PROF

10.4 离线回放能力

现状问题：

- 想重跑某个阶段，必须从头跑整条链
- 调试效率低

建议做法：

- 支持只重跑某个阶段
- 比如：只重跑本体生成、只重跑图谱构建、只重跑人设生成、只重跑报告
- 每个阶段的输入输出都保存好，可以单独重跑

10.5 分层评测体系

现状问题：

- 没有评测标准，只能靠主观感觉
- 不知道改进后效果是变好还是变差

建议做法：

- 建立最小评测集，至少覆盖：
  - 本体定义是否合理
  - 关系是否完整
  - Agent 是否符合角色
  - 模拟输出是否和预期方向一致
  - 报告是否引用到了正确事实

---

11. 后续项目推荐的落地路径

阶段一：先做结构化世界

目标：

- 跑通文档解析
- 跑通本体生成
- 跑通图谱构建

不要急着上复杂仿真，先把"现实世界 → 结构化世界"这条路走通。

阶段二：再做 Agent 化

目标：

- 从图谱筛出实体
- 生成稳定的人设
- 生成基础模拟参数

PROF

这一步主要验证"世界里的人是否成立"。

阶段三：再做演化与记忆更新

目标：

- 让 Agent 真正运行起来
- 让行为进入时序图谱
- 让检索能看到演化信息

这一步决定系统是"静态分析器"还是"动态推演器"。

阶段四：最后做报告与交互

目标：

- 用工具增强 Agent 生成报告
- 支持继续追问
- 支持采访模拟体

这样最终交付的就不是一个后台引擎，而是一个完整的可消费产品。

---

12. 一句话总结

MiroFish 最值得后续项目借鉴的，不是某一个 prompt，也不是某一个模型调用，而是这套：

分阶段构建 + 中间结果可回放 + LLM 与规则分层 + 动态图记忆 + 工具增强报告生成

的系统化设计。

如果后续项目想做得更稳，最应该优先补的是：

- **可观测性**（全链路 Trace，知道每一步在干啥）
- **分层评测**（本体、图谱、人设、模拟、报告，每层都能评）
- **Prompt 版本管理**（独立文件、版本号、可对比）
- **离线回放**（只重跑某个阶段，不用从头来）
- **更清晰的数据协议**（统一 JSON schema，模块解耦）

附录：关键代码文件索引

核心服务层（backend/app/services/）

图谱相关：

- `ontology_generator.py` - 本体生成（LLM 分析文档生成实体和关系定义）
- `graph_builder.py` - 图谱构建（动态本体生成、文本入图、Zep API 调用）
- `zep_tools.py` - GraphRAG 检索工具集（InsightForge、PanoramaSearch、QuickSearch、InterviewAgents）
- `zep_entity_reader.py` - 实体读取与筛选
- `zep_graph_memory_updater.py` - 图谱动态更新（模拟结果回写）

模拟相关：

- `simulation_manager.py` - 模拟管理（准备环境、生成配置）
- `simulation_runner.py` - 模拟运行控制
- `simulation_config_generator.py` - 模拟配置生成（LLM 智能生成时间、事件、Agent 参数）
- `oasis_profile_generator.py` - Agent 人设生成（实体 → Profile 转换）
- `simulation_ipc.py` - IPC 通信服务

报告相关：

- `report_agent.py` - ReACT 报告生成（2500+ 行，核心逻辑）
- `text_processor.py` - 文本处理（分块、预处理）

模拟脚本（backend/scripts/）

PROF

- `run_parallel_simulation.py` - 双平台并行模拟（1700+ 行，Twitter + Reddit）
- `run_twitter_simulation.py` - Twitter 单平台模拟
- `run_reddit_simulation.py` - Reddit 单平台模拟
- `action_logger.py` - 动作日志记录器（结构化日志）

API 层（backend/app/api/）

- `graph.py` - 图谱相关 API（本体生成、图谱构建、项目管理）
- `simulation.py` - 模拟相关 API（准备、启动、状态查询、Interview）
- `report.py` - 报告相关 API（生成、查询、对话）

数据模型（backend/app/models/）

- `project.py` - 项目状态管理（ProjectManager）
- `task.py` - 异步任务管理（TaskManager）

工具类（backend/app/utils/）

- `llm_client.py` - LLM 客户端封装（统一调用接口）
- `file_parser.py` - 文件解析（PDF/MD/TXT）
- `zep_paging.py` - Zep 分页查询工具
- `logger.py` - 日志工具
- `retry.py` - 重试机制

前端（frontend/src/）

视图层（views/）：

- `Home.vue` - 首页
- `MainView.vue` - 主流程视图（上传文档 → 生成本体 → 构建图谱 → 准备模拟）
- `SimulationView.vue` - 模拟配置视图
- `SimulationRunView.vue` - 模拟运行视图（实时进度）
- `ReportView.vue` - 报告展示视图（分章节流式展示）
- `InteractionView.vue` - 交互视图（对话、采访）

API 层（api/）：

- `graph.js` - 图谱 API 调用
- `simulation.js` - 模拟 API 调用
- `report.js` - 报告 API 调用

组件层（components/）：

- `HistoryDatabase.vue` - 历史项目列表
- `Step5Interaction.vue` - 交互组件

写在最后

这份文档记录了 MiroFish 项目的核心设计思路、技术难点和解决方案。

PROF

文档的价值不在于代码细节，而在于：

- 为什么这么设计（设计思路）
- 遇到了什么坑（技术难点）
- 怎么解决的（解决方案）
- 哪里还能做得更好（改进建议）

希望这份文档能帮助后续项目少走弯路，把精力放在真正有价值的创新上。

记住：好的系统不是一次性设计出来的，而是在不断解决实际问题中演化出来的。MiroFish 的这些设计，都是在真实运行中遇到问题、解决问题后沉淀下来的经验。

最后的最后：如果你要做类似的项目，建议先从小规模开始，跑通核心链路，再逐步扩展。不要一上来就想做一个完美的系统，那样很容易陷入过度设计的陷阱。